

Proyecto:	
Alumno:	Silvia García Bouza
Grupo:	DAM2 curso 2025-2026

Centro educativo: I.E.S. Fernando Wirtz Suárez

Proyecto:	
Alumno:	Silvia García Bouza
Grupo:	DAM2 curso 2025-2026

Código	Centro	Concello	Ano académico
15005397	I.E.S. Fernando Wirtz Suárez	A Coruña	2025/2026

Ciclo formativo

Código da familia profesional	Familia profesional	Código do ciclo formativo	Ciclo formativo	Grao	Réxime
FP16	Informática e comunicacións	CSINF02	Desenvolvemento Multiplataforma. Aplicacións	Superior	Dual

Módulo profesional e unidades formativas de menor duración (*)

Código MP/UF	Nome
MP0492	Proxecto de Desenvolvemento Aplicacións Multiplataforma. Equivalencia en créditos ECTS: 5. Código: MP0492. Duración: 26 horas

Profesorado responsable

Titora	Gomez Mosquera, Marian

Alumno

Alumno/a	Silvia García Bouza
-----------------	----------------------------

Datos do Proxecto

Título	Desarrollo de una solución multiplataforma para la gestión de carteras de inversión con valores reales.
---------------	--

Fecha: 25/03/2026

Nombre del alumno: Silvia García Bouza

CONTROL DE VERSIONES:

Versión	Fecha	Observaciones
2	25/03/2026	Primera revisión del proyecto

Proyecto:	
Alumno:	Silvia García Bouza
Grupo:	DAM2 curso 2025-2026

Índice:

1. Objetivo.....	5
Objetivo General:.....	5
Objetivos Específicos:.....	5
Integración de Módulos:.....	5
2. Descripción.....	6
Estudio del mercado Fintech.....	6
Fases:.....	6
Conclusión:.....	6
Motivación:.....	6
Descripción:.....	7
Reto técnico y solución:.....	7
3. Alcance.....	7
Alcance:.....	7
Límites:.....	8
4. Planificación.....	8
Viabilidad:.....	8
Análisis DAFO:.....	9
Planificación y Cronograma:.....	9
Diagrama de Gantt:.....	11
Plan de marketing:.....	11
Prevención de Riesgos:.....	11
5. Medios a utilizar.....	12
Hardware e Infraestructura:.....	12
Backend:.....	12
Frontend:.....	12
Otras Herramientas:.....	12
Recursos humanos:.....	12
6. Presupuesto.....	13
7. Título.....	14
8. Ejecución.....	14
Arquitectura de seguridad y acceso (JWT):.....	14
Desarrollo de la Persistencia y Relaciones de Datos:.....	14
Consumo de datos reales (API Externa).....	14
Validación de Datos.....	14
Seguimiento y Control de Calidad:.....	14
Uso de la aplicación:.....	15
Arranque y Autenticación (JWT).....	15

Proyecto:	
Alumno:	Silvia García Bouza
Grupo:	DAM2 curso 2025-2026

Panel Principal: Histórico:.....	15
Detalle y Gestión:.....	15
Datos Globales:.....	15
Pérdida de Conexión:.....	15
Sincronización:.....	15
9. Abstract.....	16
Project Title:.....	16
Abstract:.....	16
Key terms:.....	16
10. Diseño de la Base de Datos (Modelo Entidad-Relación).....	17
Entidades y sus Atributos:.....	17
Relaciones (Cardinalidad):.....	18
11. Repositorios.....	19

1. Objetivo

Objetivo General:

Desarrollar una solución multiplataforma denominada **InversTracking**, compuesta por una aplicación móvil y de escritorio (Flutter) y un servicio backend (Spring Boot). El sistema permitirá centralizar y monitorizar datos de carteras de inversión (acciones, criptomonedas y divisas), integrando valores de cotización reales y garantizando la persistencia híbrida (con o sin conexión al servidor) de la información.

Objetivos Específicos:

- **Desarrollo multiplataforma y correctas UI (interfaz de usuario)/UX(experiencia de usuario):** Diseñar e implementar una interfaz de usuario para móvil y escritorio mediante Flutter. El diseño se basará en las Heurísticas de Nielsen para mejorar la experiencia de usuario.
- **Arquitectura de software:** Utilizar el patrón de diseño MVVM (Model-View-ViewModel) para separar la lógica de negocio de la interfaz de usuario.
- **Gestión de datos y persistencia híbrida tanto local como remota:** Con SQLite (local) y MariaDB (remota) para tener acceso a los datos con o sin internet.
- **Comunicación y servicios:** Desarrollar un servicio backend REST API con Spring Boot que gestione la comunicación entre la aplicación y la base de datos. Además añadiré una API externa para la obtención de los datos de los valores reales de cotizaciones.
- **Seguridad y control de acceso:** Utilizando JSON Web Tokens (JWT).
- **Gestión de datos:** Gestionar los datos y generar valores de la rentabilidad de las inversiones.

Integración de Módulos:

- **Desenvolvemiento de Interfaces:** Diseñaré e implementaré una interfaz multiplataforma con Flutter. El diseño será responsivo y seguiré las Heurísticas de Nielsen. Incluiré soporte multi-idioma (Castellano/Galego).
- **Programación Multimedia e Dispositivos Móviles:** Adoptaré la arquitectura MVVM para separar la lógica de la interfaz. Gestionaré el ciclo de vida de la aplicación y la comunicación con la Api externa para obtener los valores reales. Además, desarrollaré un sistema de persistencia híbrida (con y sin conexión al servidor).
- **Acceso a Datos:** Configuraré SQLite para el almacenamiento local. Realizaré la sincronización remota mediante una base de datos MAriaDB. Crearé un backend con Spring Boot para gestionar servicios REST.
- **Programación de Servicios e Procesos:** Utilizaré programación asíncrona para las llamadas a la API externa y garantizaré la seguridad mediante protocolos HTTPS y autenticación con JWT.

- **Sistemas de Xestión Empresarial:** Implementaré funcionalidades para la gestión administrativa de los datos y la lógica necesaria para generar valores sobre la rentabilidad de las carteras.
- **Empresa e Iniciativa Emprendedora:** Elaboraré un plan de negocio completo que incluirá el análisis de viabilidad, estudio de mercado Fintech, análisis DAFO y la planificación temporal del desarrollo (Gantt).
- **Inglés Profesional:** Redactaré el Abstract técnico y prepararé la defensa oral del proyecto.

2. Descripción

Estudio del mercado Fintech

Fases:

- **Análisis de demanda y puntos débiles:** Identificación del desconocimiento del estado real de la inversión provocada por la dispersión de las inversiones en muchas plataformas.
- **Segmentación del ecosistema Fintech:** Mapeo de exchanges, brokers y bancos para posicionar a InversTracking como una aplicación unificadora.
- **Benchmarking de soluciones de agregación:** Evaluación de la competencia para destacar este modelo no custodio frente al estándar bancario tradicional.
- **Validación de soberanía de datos:** Análisis de la percepción de seguridad del usuario al trackear su patrimonio sin exponer claves ni fondos reales.
- **Estudio de viabilidad regulatoria:** Verificación del cumplimiento legal al operar como una herramienta de solo lectura y gestión privada.
- **Estrategia de posicionamiento:** Lanzamiento de una solución que devuelve el poder y la privacidad total a la persona usuaria.

Conclusión:

InversTracking soluciona la fragmentación financiera actual unificando activos dispersos en un solo lugar. Mi proyecto destaca por:

- **Modelo no custodio:** Es una aplicación solo de información y visualización.
- **Soberanía de datos:** El usuario es el único dueño de su información, garantizando una privacidad que las plataformas tradicionales no ofrecen.

Motivación:

En el contexto financiero actual, los inversores suelen diversificar su capital en múltiples plataformas, como exchanges de criptomonedas, brokers de acciones y banca tradicional. Esta fragmentación dificulta obtener una visión clara y rápida de la rentabilidad total de las inversiones. La motivación de este proyecto es desarrollar una herramienta que

permita al usuario centralizar toda su información financiera de manera privada y eficiente en un único lugar.

Descripción:

Desarrollaré una aplicación multiplataforma compuesta por un cliente en Flutter y un servidor con Spring Boot. La aplicación permitirá al usuario registrarse de forma segura para gestionar su cartera personal de inversiones de manera integral. Implementaré funcionalidades para dar de alta, y eliminar activos (acciones, criptomonedas y divisas), permitiendo al sistema calcular automáticamente la rentabilidad total mediante la integración con APIs externas. El usuario podrá consultar el histórico de sus operaciones. Todo ello con una interfaz responsiva que garantiza la misma experiencia de uso en dispositivos móviles y de escritorio, con soporte completo en castellano y gallego.

Reto técnico y solución:

El mayor desafío técnico del proyecto reside en garantizar la disponibilidad de los datos mediante una estrategia de persistencia híbrida. Implementaré un sistema de almacenamiento local con SQLite para asegurar que el usuario tenga acceso inmediato a su cartera de activos, permitiendo la consulta de datos incluso si se pierde la conexión con el servidor. Solucionaré este reto creando un sistema que compare los datos guardados en local con los del servidor en cuanto se recupere la conexión. Además, usaré tokens de seguridad (JWT) para que este intercambio de información sea privado y solo el usuario pueda ver sus datos.

3. Alcance

Alcance:

- **Gestión de cartera:** Permitiré que el usuario pueda añadir, ver y eliminar activos financieros en tres categorías principales: acciones, criptomonedas y divisas.
- **Seguimiento de transacciones:** Se podrán ver las transacciones que se fueron realizando.
- **Cálculo de rentabilidad con valores reales:** Integraré una API externa para obtener cotizaciones actuales, para calcular automáticamente la ganancia o pérdida de cada activo y del total de la cartera.
- **Visualización de métricas:** Habrá una pantalla para visualizar el rendimiento total de los activos.
- **Modo desconectado y sincronización:** El usuario podrá ver, añadir y eliminar los datos sin conexión a la base de datos, sincronizándolos con el servidor cuando se recupere el acceso.

Límites:

- **No es una plataforma de trading:** La aplicación es exclusivamente informativa y de gestión; no realizará compras o ventas en mercados reales ni actuará como broker.
- **No es una cartera de custodia:** El sistema no almacenará claves privadas de criptomonedas ni fondos reales. El usuario deberá introducir sus posiciones de forma manual.
- **Sin asesoramiento financiero:** La aplicación muestra métricas y datos de rentabilidad. No ofrecerá recomendaciones de inversión ni análisis predictivos basados en inteligencia artificial en esta fase inicial.

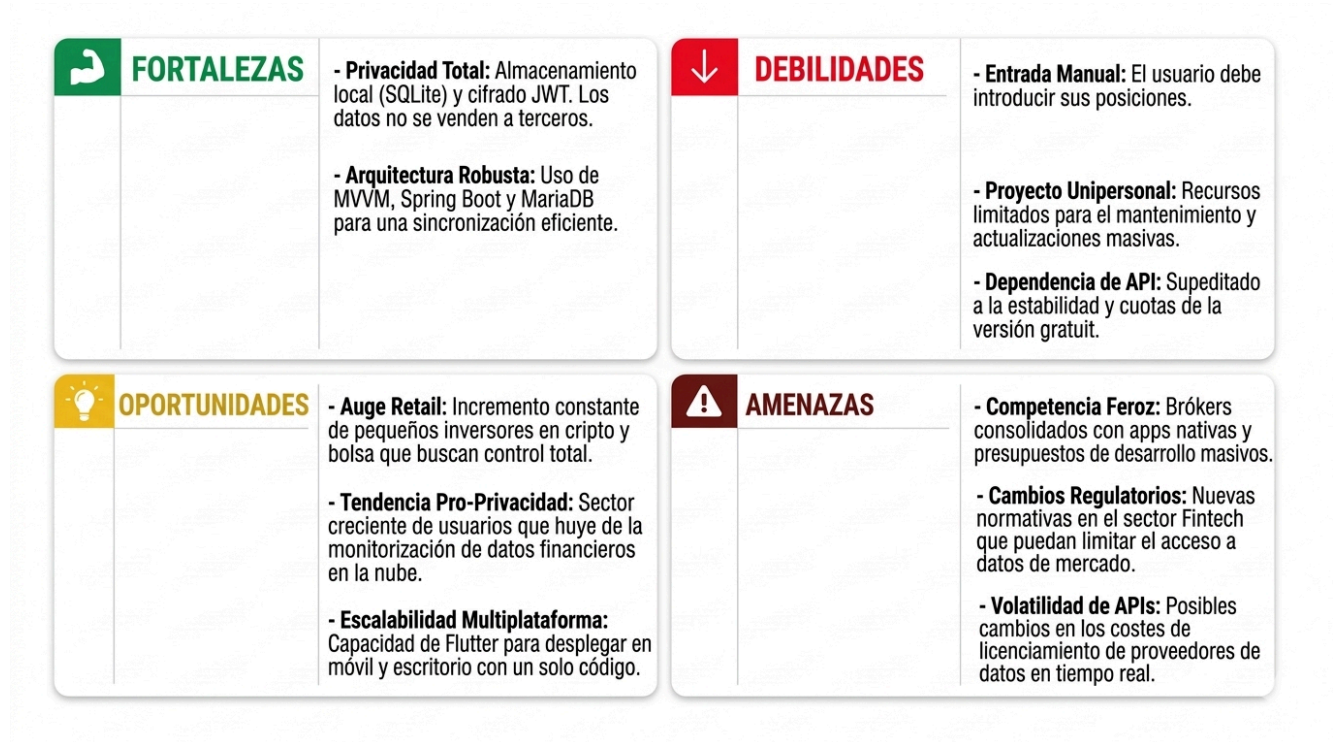
El impacto de estos límites es positivo para la viabilidad del proyecto, ya que permite centrar el trabajo en crear una aplicación únicamente de gestión de inversiones. Al eliminar la complejidad legal y técnica de la custodia de fondos o el trading, aseguro que el proyecto sea realizable en los plazos establecidos.

4. Planificación

Viabilidad:

- **Técnica:** Cuento con los conocimientos necesarios en el stack tecnológico propuesto (Flutter y Spring Boot) para llevar a cabo el desarrollo integral del sistema. La elección de una tecnología multiplataforma me permite optimizar los tiempos de implementación al compartir una única base de código para móvil y escritorio.
- **Económica:** Aunque el proyecto no requiere una inversión de capital externo inicial al utilizar tecnologías Open Source (MariaDB, SQLite, Spring Boot), el mayor coste es el tiempo dedicado.
- **Seguridad:** He diseñado el proyecto siguiendo la normativa de protección de datos y seguridad técnica, minimizando los riesgos legales al ser una herramienta informativa que no gestiona activos reales.

Análisis DAFO:



Planificación y Cronograma:

La ejecución del proyecto se distribuye en 6 etapas de trabajo, organizadas de la siguiente manera:

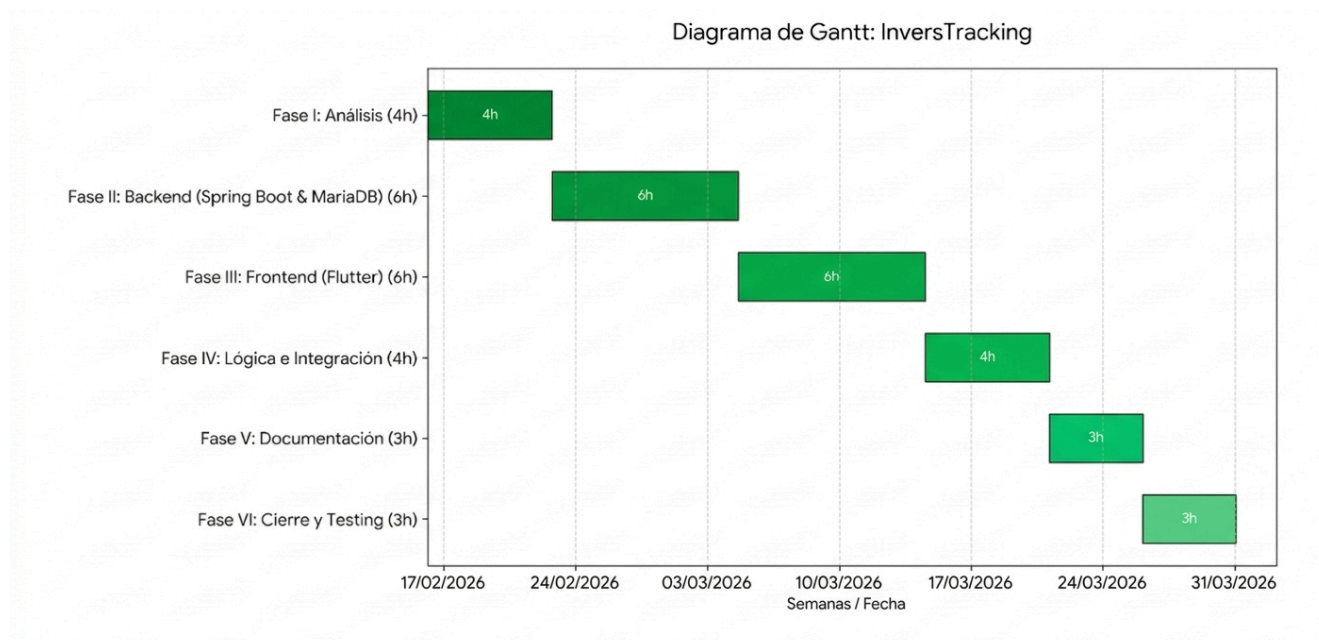
Fase	Tareas Principales	Tiempo	Recursos y Permisos
I. Análisis	Diseño de la base de datos híbrida (MariaDB/SQLite).	4h	DBeaver, Esquema ER.
II. Backend	Construcción de API REST y seguridad JWT con Spring Boot.	6h	IntelliJ, Java, MariaDB.

Proyecto:	
Alumno:	Silvia García Bouza
Grupo:	DAM2 curso 2025-2026

III. Frontend	Desarrollo de pantallas y navegación en Flutter.	6h	VS Code, Flutter SDK.
IV. Lógica	Integración con API externa y cálculos de rentabilidad.	4h	Permiso: API Key.
V. Documentación	Documentación de código, README.	3h	MarckDown, Comentarios.
VI. Cierre	Testing final, depuración de errores y entrega.	3h	Portátil y móvil..

He decidido comenzar por el diseño de la base de datos y la construcción del Backend (Fases I y II) antes de iniciar el Frontend. Esta decisión me permite tener una estructura de datos y una API funcional antes de empezar el desarrollo en Flutter. De esta manera, el cliente Flutter siempre tendrá servicios estables que consumir durante su desarrollo, minimizando riesgos de rediseño.

Diagrama de Gantt:



Plan de marketing:

Lanzaré **InversTracking** pensando en personas que, como yo, valoran su privacidad por encima de todo. Mi mensaje principal será que la aplicación no toca el dinero, solo lo ordena, lo que da una seguridad que otras plataformas no ofrecen.

Para llegar a los usuarios:

- **Tiendas de aplicaciones:** Usaré palabras clave de inversión y cripto para que me encuentren fácilmente.
- **Grupos de Telegram:** Entraré en comunidades donde la gente tiene sus ahorros repartidos en múltiples sitios para enseñarles cómo juntarlo todo de forma privada.

Prevención de Riesgos:

En este proyecto tendré en cuenta dos tipos de riesgos:

- **Riesgos laborales:** Al pasar 26 horas frente al ordenador, cuidaré la ergonomía para evitar lesiones.
- **Riesgos lógicos:** El mayor riesgo es la pérdida de datos o del código fuente. Para prevenirlo, usaré GitHub.

5. Medios a utilizar

Hardware e Infraestructura:

- **Ordenador portátil:** Para el análisis, desarrollo y pruebas.
- **Conexión a internet:** Necesaria para la consulta de documentación, descarga de dependencias y consumo de la API externa.

Backend:

- **IDE:** IntelliJ IDEA (Community Edition - Licencia gratuita).
- **Lenguaje y framework:** Java con Spring Boot.
- **Persistencia remota:** MariaDB (Open Source).
- **Seguridad:** Implementación de JSON Web Tokens (JWT) para la autenticación.

Frontend:

- **IDE:** Visual Studio Code (Open Source).
- **Lenguaje y framework:** Dart y Flutter.
- **Persistencia local:** SQLite.

Otras Herramientas:

- **Control de versiones:** Git y GitHub.
- **Gestión de bases de datos:** DBeaver.
- **Pruebas de API:** Postman para la validación de los endpoints del backend.
- **API externa:** Para recuperar los datos de los valores de los activos.

Recursos humanos:

Yo misma diseñaré, desarrollaré y testearé la aplicación.

Tiempo estimado: 26 horas.

6. Presupuesto

Concepto	Detalle	Total (est.)
Mano de Obra	26h x 50,00 €/h (Análisis, desarrollo y testing)	1.300,00 €
Licencias Software	Herramientas Open Source (Spring Boot, Flutter, MariaDB)	0,00 €
Infraestructura	Uso de hardware propio y entornos locales	0,00 €
Subtotal		1.300,00 €
IVA (21%)		273,00 €
TOTAL		1.573,00 €

Presupuesto total estimado: 1573€

En cuanto a la financiación, el proyecto está financiado por mí misma.

- Inversión en Software: 0,00 €, al apoyarme exclusivamente en tecnologías de código abierto (MariaDB, SQLite, Spring Boot, Flutter).
- Capital Humano: La inversión principal reside en mi propia dedicación como desarrolladora (valorada en el presupuesto previo de 1.573,00 €).
- Infraestructura: Uso de hardware propio para el desarrollo y despliegue inicial en servidores locales, eliminando la necesidad de financiación externa en esta fase MVP (Producto Mínimo Viable).

7. Título

Desarrollo de una solución multiplataforma para la gestión de carteras de inversión con valores reales.

8. Ejecución

Arquitectura de seguridad y acceso (JWT):

He asegurado el acceso con JWT. El servidor genera un token que la aplicación envía en cada consulta. He configurado un filtro que la revisa al momento, permitiendo el acceso solo si es correcta o no ha caducado.

Desarrollo de la Persistencia y Relaciones de Datos:

Utilizaré una arquitectura de persistencia híbrida que combina SQLite en el dispositivo y MariaDB con JPA/Hibernate en el servidor. He diseñado las relaciones entre items y transacciones de forma bidireccional para mantener las tablas limpias, aplicando FetchType.LAZY para que los datos solo se carguen cuando sea necesario y no sobrecargar la memoria. Además, usaré CascadeType.ALL para que, al borrar un activo, sus movimientos se eliminen automáticamente, manteniendo la base de datos siempre saneada y sin registros huérfanos.

Consumo de datos reales (API Externa)

Integraré una API externa para obtener precios reales y calcularé automáticamente la rentabilidad comparando el historial de compras con el mercado.

Validación de Datos

Para que la base de datos no se llene de fallos, voy a validar todos los datos en los formularios de la aplicación antes de enviarlos. Me aseguraré de que no se queden campos obligatorios vacíos y que los precios o cantidades tengan sentido (que no sean negativos, por ejemplo).

Seguimiento y Control de Calidad:

- Comprobaré que la aplicación cargue los datos en pocos segundos, que la interfaz sea fácil de usar (usabilidad) y que el sistema de seguridad JWT no deje pasar a nadie sin permiso.

- Si encuentro un fallo o "bug" durante el desarrollo, lo anotaré en un registro y lo arreglaré según su importancia: primero los fallos que impidan el funcionamiento de la aplicación y después otros detalles menos funcionales.
- Testear la aplicación.

Uso de la aplicación:

Arranque y Autenticación (JWT)

- **Acción:** Abro mi aplicación e introduzco mis credenciales.
- **Proceso:** Mi backend en Spring Boot valida los datos. Si soy yo, mi JwtService me genera un token . Si fallo, mi JwtAuthenticationEntryPoint me bloquea con un error 401 "No autorizado".

Panel Principal: Histórico:

- **Acción:** Entro al listado de transacciones. La app hace un GET al backend.
- **Visualización:** Veo la lista de transacciones cargada desde la base de datos.

Detalle y Gestión:

- **Detalles:** Clicko en una transacción y se abre una pantalla con toda la información de esa transacción
- **Eliminar:** Pulso el icono de papelera en una fila. La aplicación lanza un DELETE al servidor y refresca la lista al instante.
- **Añadir:** Pulso el botón +. Relleno los datos de la nueva transacción y se guarda en la base de datos.

Datos Globales:

- **Acción:** Pulso el botón para ver mis datos generales.
- **Resultado:** Veo el resumen de mi patrimonio total y rentabilidad acumulada.

Pérdida de Conexión:

- **Acción:** Te quedas sin conexión al servidor.
- **Uso:** Sigo viendo mi histórico, puedo añadir nuevas compras o borrar activos. Todo se guarda local.

Sincronización:

- **Acción:** Vuelve la conexión. Pulso el botón para sincronizar.
- **Ejecución:** Envío mis cambios locales al servidor para actualizar mi base de datos en MariaDB.

9. Abstract

Project Title:

Development of a multi-platform application for managing investment portfolios with real-market data.

Abstract:

InversTracking is a multi-platform application designed to help people manage their different investments in one place. Nowadays, many investors have their money in many different platforms, such as banks, stock brokers, or crypto exchanges. This makes it very difficult to have a clear view of their total wealth. This project offers a solution developed with Flutter for the mobile and desktop app, and Spring Boot for the server. It allows users to track their net worth and transaction history easily, without the need to move real funds or share private security keys.

From a technical point of view, the application uses a hybrid system to save data. It uses SQLite for offline access on mobile devices and MariaDB to synchronize information with the cloud when an internet connection is available. Security is a very important part of the project, so we use JSON Web Tokens (JWT) and Spring Security to protect the API and ensure that only authorized users can access their data. Additionally, the user interface follows Nielsen's Heuristics to be simple and friendly, including animations and audio to improve the experience. The main goal is to give the user privacy and full control over their financial information. Finally, the application supports both Spanish and Galician languages to help local users in their own language.

Key terms:

Portfolio Management, Flutter, REST API, Asynchronous Programming, Data Privacy.

10. Diseño de la Base de Datos (Modelo Entidad-Relación)

Para guardar toda la información en el servidor he configurado una base de datos con MariaDB. He diseñado las tablas de forma que la información esté organizada y conectada, evitando que los datos se dupliquen o que haya errores con los cálculos de las inversiones.

Entidades y sus Atributos:

```
MariaDB [invest_db]> SHOW TABLES;
+-----+
| Tables_in_invest_db |
+-----+
| categories            |
| items                 |
| transactions          |
| users                 |
+-----+
4 rows in set (0,001 sec)
```

-users (Usuarios):

- id (PK): Identificador único (Autoincremental).
- username: Nombre de usuario (Único).
- password: Contraseña.
- email: Correo electrónico. Puede ser nulo.

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression	Comment
123 id	1	bigint(20)	[V]	[V]	PRI		auto_increment		
A-Z username	2	varchar(255)	[V]	[]	UNI				
A-Z password	3	varchar(255)	[V]	[]					
A-Z email	4	varchar(255)	[]	[]		NULL			

- categories (Categorías): Define el tipo de activo.

- id (PK): Identificador único.
- name: Nombre de la categoría.

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression	Comment
123 id	1	bigint(20)	[V]	[V]	PRI		auto_increment		
A-Z name	2	varchar(255)	[V]	[]					

- items (Activos): El activo específico que el usuario tiene.

- id (PK): Identificador único.
- name: Nombre del activo.

- user_id (FK): Relación con el usuario.
- category_id (FK): Relación con su categoría.

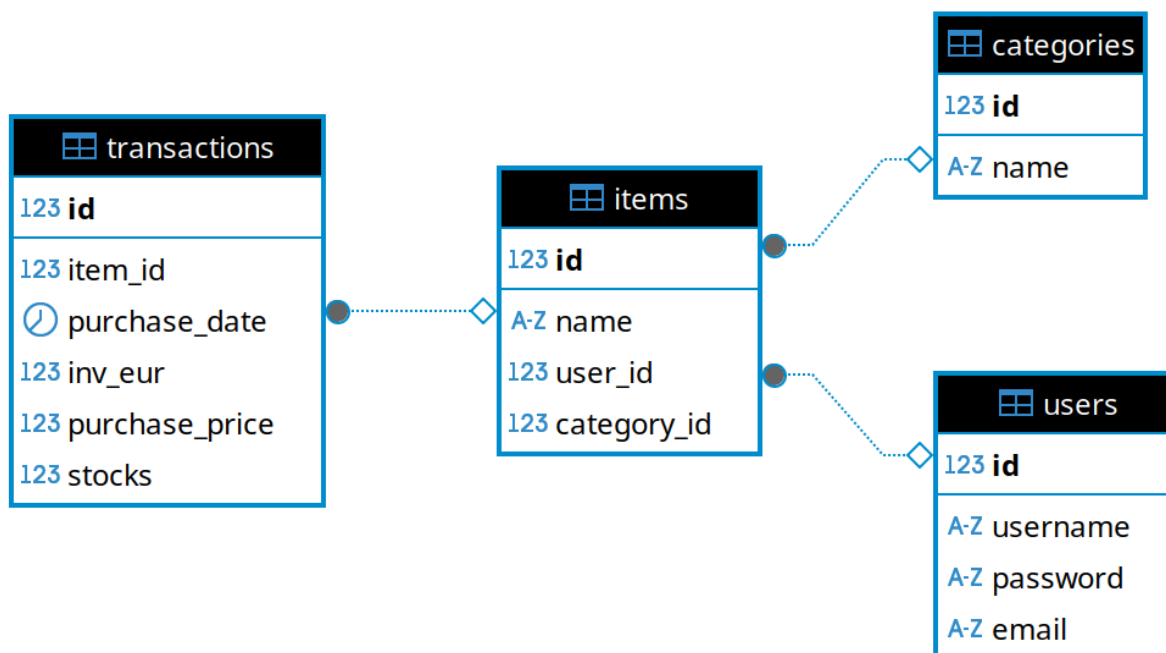
Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression	Comment
123 id	1	bigint(20)	[V]	[V]	PRI		auto_increment		
A-2 name	2	varchar(255)	[V]	[]					
123 user_id	3	bigint(20)	[]	[]	MUL	NULL			
123 category_id	4	bigint(20)	[]	[]	MUL	NULL			

- **transactions (Transacciones):** El historial de compras.
 - id (PK): Identificador único.
 - item_id (FK): Relación con el Item.
 - stocks: Cantidad de unidades.
 - purchase_price: Precio por unidad al momento de compra.
 - inv_eur: Inversión total en euros.
 - purchase_date: Fecha y hora de la operación.

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression	Comment
123 id	1	bigint(20)	[V]	[V]	PRI		auto_increment		
123 item_id	2	bigint(20)	[]	[]	MUL	NULL			
123 purchase_date	3	datetime	[V]	[]					
123 inv_eur	4	double	[V]	[]					
123 purchase_price	5	double	[V]	[]					
123 stocks	6	double	[V]	[]					

Relaciones (Cardinalidad):

Relación	Tipo	Descripción
User — Item	1 : N	Un usuario puede tener múltiples items (activos) en su cartera, pero un item pertenece a un único usuario.
Category — Item	1 : N	Una categoría puede englobar muchos items , pero cada item tiene una sola categoría.
Item — Transaction	1 : N	Un ítem puede tener muchas transacciones de compra, pero una transacción es de un solo item.



11. Repositorios

Actualmente estoy desarrollando el proyecto.

Dejo los enlaces a mis repositorios de GitHub:

- Backend: https://github.com/SilviaGarciaBouza/Backend_InvestMent_Tracking
- Fronted: https://github.com/SilviaGarciaBouza/Investment_Tracking